

0.87.4. Agrandir une lettre : `texteGrand` — la plus grande taille, 10

La plus grande taille : `texteGrand(0, 0, "A", 10)`. Le A est 11 fois plus grand : 11×11 cases, plus de la moitié de l'écran.

1. Ce qu'il faut comprendre

La taille la plus grande, 10 : $10 + 1 = 11$ fois plus grand ; chaque pixel du A devient un carré de 11×11 pixels. La lettre prend **11×11 cases** (88×88 pixels) : plus de la moitié de l'écran.

Pourquoi 10 au plus : les tailles vont de 0 à 10, comme une note sur 10. Au-delà, le compilateur refuse, et dit pourquoi : « la taille va de 0 à 10 ».

Pour tenir : 11 cases de large sur 20, 11 de haut sur 18. Le A géant commence en (0, 0), il reste de la place à droite et en dessous.

Ça ne coûte presque rien en dessins : à cette taille, beaucoup de tuiles sont toutes pleines ou toutes vides, et les tuiles identiques ne sont fabriquées qu'une fois.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texteGrand>` → [0.86.3. La fonction `texteGrand\(\)` — un texte agrandi](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1912 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage           55 o ← la console, à l'allumage — la mise en route de la console
texte_grand_0()      538 o ← main() — ta fonction
main()              16 o ← démarrage — ta fonction
VBlank              8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage       28 o ← image() — attend l'image suivante
AttendreVBlank      16 o ← démarrage — attend que l'écran soit libre
AdresseCase        10 o ← poser() — trouve l'adresse d'une case de l'écran
CopierTuiles        18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage — vide les deux cartes de l'écran
Tuiles             1088 o ← CopierTuiles — la police entière, et 24 tuiles dessinées
GRAND_TABLE_0_0_0    11 o ← poser() — une table gravée
GRAND_TABLE_0_1_1    11 o ← poser() — une table gravée
GRAND_TABLE_0_2_2    11 o ← poser() — une table gravée
GRAND_TABLE_0_3_3    11 o ← poser() — une table gravée
GRAND_TABLE_0_4_4    11 o ← poser() — une table gravée
GRAND_TABLE_0_5_5    11 o ← poser() — une table gravée
GRAND_TABLE_0_6_6    11 o ← poser() — une table gravée
GRAND_TABLE_0_7_7    11 o ← poser() — une table gravée
GRAND_TABLE_0_8_8    11 o ← poser() — une table gravée
GRAND_TABLE_0_9_9    11 o ← poser() — une table gravée
```

```

GRAND_TABLE_0_10_10 11 o ← poser() – une table gravée
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre,
Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond,
EffacerPanneau, LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()          × 11  339 o sur place, dans texte_grand_0() + AdresseCase (10 o), GRAND_TABLE_0_0_0 (11 o),
GRAND_TABLE_0_1_1 (11 o), GRAND_TABLE_0_2_2 (11 o), GRAND_TABLE_0_3_3 (11 o), GRAND_TABLE_0_4_4 (11 o),
GRAND_TABLE_0_5_5 (11 o), GRAND_TABLE_0_6_6 (11 o), GRAND_TABLE_0_7_7 (11 o), GRAND_TABLE_0_8_8 (11 o),
GRAND_TABLE_0_9_9 (11 o), GRAND_TABLE_0_10_10 (11 o)
  image()          × 1   3 o sur place, dans main() + AttendreImage (28 o)
Tes fonctions appelées :
  texte_grand_0() × 1   3 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <texteGrand> // écrit un texte agrandi, taille de 0 à 10
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

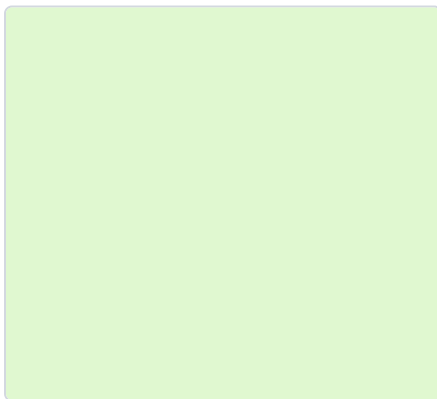
int main() {
  // La plus grande taille : 10. Le A est 11 fois plus grand : 11 × 11 cases.
  // 8 pixels × 11 = 88 de large
  // 7 pixels × 11 = 77 de haut
  texteGrand(0, 0, "A", 10);

  while (true) {
    image();
  }
}

```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A immense en haut à gauche, sur plus de la moitié de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A géant remplit une bonne part de ses 11×11 cases (59 cases pleines)

✓ il ne déborde pas

Ensuite : **0.87.5. La fonction `texteGrandS()` — un grand texte qui va à la ligne** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++